Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



A Report on Lab-06

Design and Development of a Modular Library and Fine Management System Following the Full Software Development Lifecycle (SDLC)

Course Code: CSE 3206
Course Title: Software Engineering Sessional

Submitted By

Tanzirul Islam
Roll No.: 2203054
Section: A

Submitted To

Emrana Kabir Hashi
Assistant Professor
Department of Computer Science & Engineering
Rajshahi University of Engineering & Technology

Date: 31 Oct. 2026

Project Repository: https://github.com/TanzirulIslam22/BookWise

---

1 Introduction and Problem Statement

Academic libraries frequently manage resource circulation and overdue fines using disjointed spreadsheets or physical registers. Manual workflows regularly result in inaccurate book availability counts, overdue date miscalculations, inconsistent fine enforcement across member categories, and an absence of verifiable transaction histories.

The **Smart Library & Fine Management System** provides a robust, testable, and maintainable software solution that automates library operations across five core functional modules:

1. **Member Management:** Manages member profiles, unique institutional identifiers, and role-based permissions (Student, Faculty, Librarian).
2. **Book Catalog:** Tracks titles, authors, unique ISBNs, total acquisitions, and live available copy counts.
3. **Borrow & Return Processing:** Coordinates loans with 14-day return horizons, enforces borrowing limits per role, and restores inventory upon return.
4. **Fine Calculation Management:** Dynamically computes progressive overdue fines using polymorphic role policies.
5. **Audit & Notification Logging:** Records immutable system event logs and surfaces non-blocking contextual feedback in the client interface.

Data persistence is implemented using MongoDB with Mongoose object data modeling.

2 Requirements and Process Model

2.1 Functional Requirements

Table 1 shows the main functional requirements of the system. Each requirement is checked by unit or integration tests.

2.2 Mathematical Model for Overdue Fines

Let d be the number of days after the due date. The fine for a student is:

$$\text{Fine}_{\text{Student}}(d) = \begin{cases} 0, & d \leq 0 \\ 10 \times d, & 1 \leq d \leq 7 \\ 70 + 20 \times (d-7), & d > 7 \end{cases} \tag{1}$$

For faculty, $\text{Fine}_{\text{Faculty}}(d) = 0$ for all $d \geq 0$.

Table 1: Summary of Functional Requirements

Module	Requirements
Members	FR-1.1: Register with unique member ID. FR-1.2: Reject duplicate IDs with HTTP 409. FR-1.3: Validate name and role. FR-1.4: Real-time search and filter.
Books	FR-2.1: Register book with unique ISBN and copy count. FR-2.2: Reject duplicates. FR-2.3: Search by title, author, and ISBN. FR-2.4: Available copies must stay between 0 and totalCopies.
Borrow / Return	FR-3.1: Issue loans with due date = today + 14 days. FR-3.2: Borrow limit (Student: 3, Faculty: 5, Librarian: 0). FR-3.3: Block borrowing when no copy is left. FR-3.4: Increase available copies on return.
Fines	FR-4.1: Calculate fine from whole overdue days ($d \geq 0$). FR-4.2: Student rate is 10 BDT/day. FR-4.3: After day 7, rate becomes 20 BDT/day. FR-4.4: No fine for faculty. FR-4.5: Live overdue fine preview.
Audit Log	FR-5.1: Save a log on issue, return, and fine events. FR-5.2: Show toast notification in the app. FR-5.3: Show logs in time order.

2.3 Process Model

We used the **Incremental Model** because each part of the system depends on the previous one. Books and members had to work properly before borrowing and fines could be added. The project was built in four steps:

- **Increment 1:** Database setup, member registration, and book management.
- **Increment 2:** Borrow and return, borrowing limits, and stock update.
- **Increment 3:** Fine calculation by role, and activity logging.
- **Increment 4:** Automated tests, Swagger API docs, CI pipeline, and bug fixes.

So the final system is the sum of all increments:

$$\text{System} = I_1 + I_2 + I_3 + I_4 \tag{2}$$

In this way, every part was tested and working before the next part was added.

3 System Architecture and Design Patterns

The application is built using a standard **3-Tier Architecture** to maintain a clean separation of concerns:

- **Presentation Tier (Frontend):** A responsive React Single Page Application (SPA).
- **Application Tier (Backend):** A Node.js and Express.js RESTful API handling business logic.
- **Data Tier (Database):** A MongoDB database managing persistent document storage.

The core request flow through the system layers is:

React Client ⟶ Express Controllers ⟶ BorrowFacade ⟶ Domain Services ⟶ MongoDB

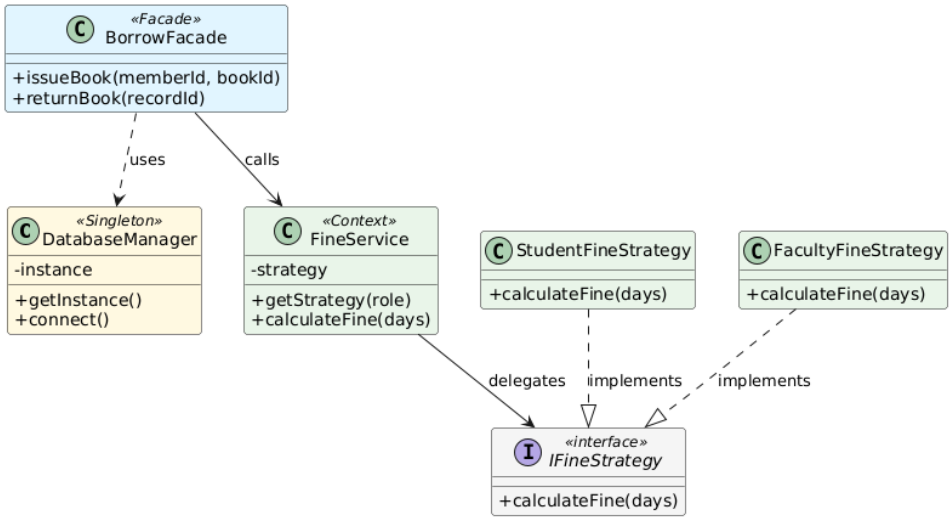


Figure 1: UML Class Diagram highlighting Core Design Patterns

3.1 Design Pattern Implementations

As illustrated in Figure 1, three core Gang-of-Four (GoF) design patterns were implemented to keep the codebase maintainable and modular:

1. **Singleton Pattern (Creational):**
   - **Implementation:** `DatabaseManager`
   - **Problem Solved:** Opening multiple database connections causes high memory overhead. The Singleton pattern ensures that only one database instance is created and shared across the entire application lifecycle.
   - **Why Chosen:** Prevents redundant database connections and keeps connection pooling centralized and stable.
2. **Facade Pattern (Structural):**
   - **Implementation:** `BorrowFacade`
   - **Problem Solved:** Borrowing or returning a book involves multiple independent actions (checking member limits, verifying stock, calculating due dates, and logging events). The Facade pattern hides this complexity behind two simple methods: `issueBook()` and `returnBook()`.
   - **Why Chosen:** Simplifies the controller code and protects the presentation layer from dealing directly with internal service details.
3. **Strategy Pattern (Behavioral):**

- **Implementation:** `FineService` with `IFineStrategy` (`StudentFineStrategy`, `FacultyFineStrategy`)
- **Problem Solved:** Different user roles follow different fine rules (Students pay a tiered rate after 7 days, whereas Faculty members pay zero fine). The Strategy pattern separates each calculation rule into its own class.
- **Why Chosen:** Makes it easy to change or add new fine policies in the future without writing messy `if-else` blocks, following the Open/Closed Principle.

4 Software Testing and Verification

To ensure application stability and correctness, automated testing was conducted across unit, integration, and end-to-end levels using Jest, Supertest, and Selenium.

4.1 Test-Driven Development (TDD)

The core business logic, particularly the fine calculation engine, was developed following the standard **Test-Driven Development (TDD)** workflow:

1. **Red Phase:** Unit tests were written first specifying the expected fine amounts for on-time and overdue returns. As expected, tests failed initially because the implementation did not exist yet.
2. **Green Phase:** Minimal business logic was implemented in `FineService` until all written test cases passed successfully.
3. **Refactor Phase:** The code was cleaned and separated into role-specific strategies (`StudentFineStrategy` and `FacultyFineStrategy`) while ensuring all tests continued to pass.

4.2 Mocking External Dependencies

To test components in complete isolation, external dependencies such as the database and logging services were mocked using Jest's built-in mocking utilities (`jest.fn()`):

- **Database Isolation:** Mongoose database calls were mocked in unit tests so that test cases execute quickly in memory without writing dummy data to a live database.
- **Integration Testing:** For API route testing, Supertest was used alongside an in-memory database (`mongodb-memory-server`) to safely verify HTTP responses (such as status codes 200, 400, and 409).

4.3 Test Coverage Summary

The test suite was executed using Jest to measure code coverage across all core application layers. As summarized in Table 2, the project achieved an overall statement coverage of **89.77%**

and a line coverage of **90.88%**, confirming that core domain paths and error handlers are thoroughly verified.

Table 2: Automated Test Coverage Summary (Jest)

Component Module	% Statements	% Branches	% Functions	% Lines
`DatabaseManager.js`	82.60	80.00	80.00	82.60
`BorrowFacade.js`	95.08	82.85	100.00	95.08
`FineService.js`	95.65	90.90	100.00	95.45
`AuditService.js`	85.71	33.33	100.00	85.71
`Fine Strategies`	93.75	90.47	83.33	93.75
`Middleware & Models`	94.33	70.45	100.00	94.16
Overall Coverage	**89.77%**	**73.33%**	**91.01%**	**90.88%**

4.4 Automated End-to-End (E2E) UI Testing

In addition to backend unit tests, an automated End-to-End browser test was created using **Selenium WebDriver**. Running in headless Chrome, the test automatically simulates a full student journey: logging into the dashboard, issuing a book, verifying that available stock decreases by 1, and confirming the return process.

5 Configuration Management (Git/GitHub)

5.1 Feature Branching and Pull Request Reviews

Development strictly followed an organized feature branching workflow (`feature/member-management`, `feature/borrow-return`, and `feature/fine-strategy`) to prevent pushing untested code directly into the `main` production branch.

Collaborative review practices were demonstrated through two pull requests:

- **Accepted Pull Request:** A PR for the fine calculation strategy was opened from `feature/fine-strategy`, reviewed against test specifications, and successfully merged into `main` (Figure 2).
- **Rejected/Revised Pull Request:** A PR introducing an experimental multiplier on `feature/experimental-penalty` received change requests during review and was closed without merging (Figure 3) to safeguard fine boundary rules.

5.2 Merge Conflict Creation and Resolution

To demonstrate conflict resolution proficiency, a merge conflict was simulated by updating the release version line in `README.md` simultaneously from two concurrent branches (`branch-a` and `branch-b`).

Attempting to merge `branch-b` into `main` produced standard Git conflict markers (`<<<<<< HEAD`, `=======`, `>>>>>> branch-b`) as shown in Figure 4. The conflict was manually resolved in the

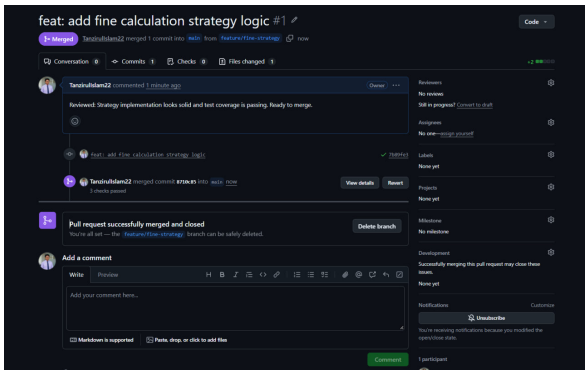


Figure 2: Accepted and Merged Pull Request

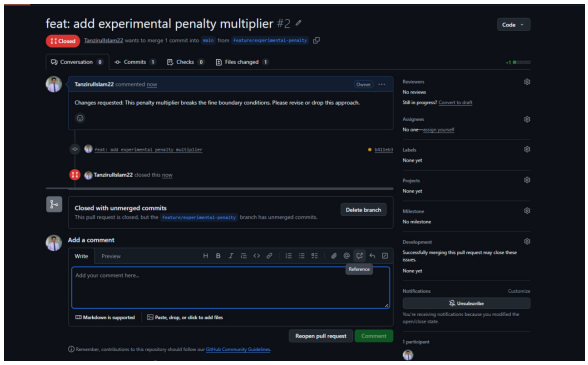


Figure 3: Closed/Rejected Pull Request

editor by retaining the production release line, staging the file, and committing the fix with a descriptive message (Figure 5).

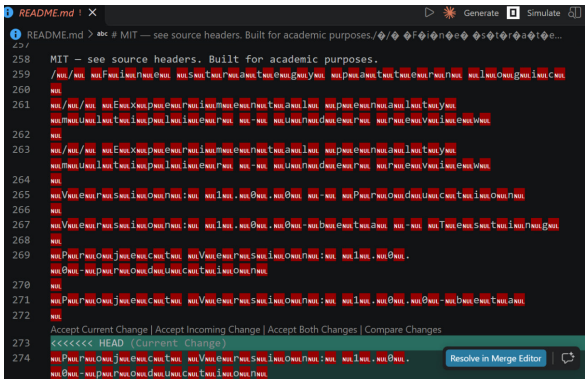


Figure 4: Merge Conflict Markers in Code Editor

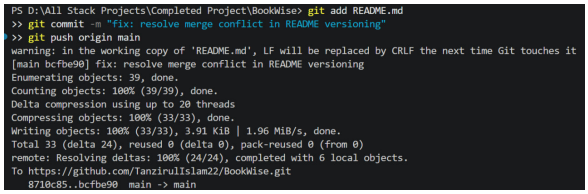


Figure 5: Conflict Resolution and Final Commit

5.3 Continuous Integration (GitHub Actions)

A Continuous Integration (CI) pipeline was established using GitHub Actions (`.github/workflows/ci.yml`) to automate verification on every push and pull request. As shown in Figure 6, the pipeline automatically runs backend test suites against an in-memory database and builds the frontend bundle, with all checks passing successfully.

6 New Tool or Technique (Swagger / OpenAPI)

6.1 Interactive API Documentation with Swagger UI

As a self-directed technology learned outside the sessional curriculum, the backend integrates the **OpenAPI 3.0 specification** served via `swagger-ui-express` at the `/api-docs` endpoint.

Unlike static API tables or external Postman collections, Swagger UI allows developers to view request schemas, test live endpoints with interactive parameters, and observe real-time status codes directly from the browser as shown in Figure 7.

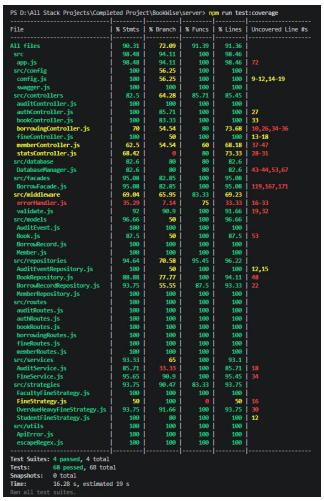


Figure 6: Automated GitHub Actions CI Pipeline with Passing Checks

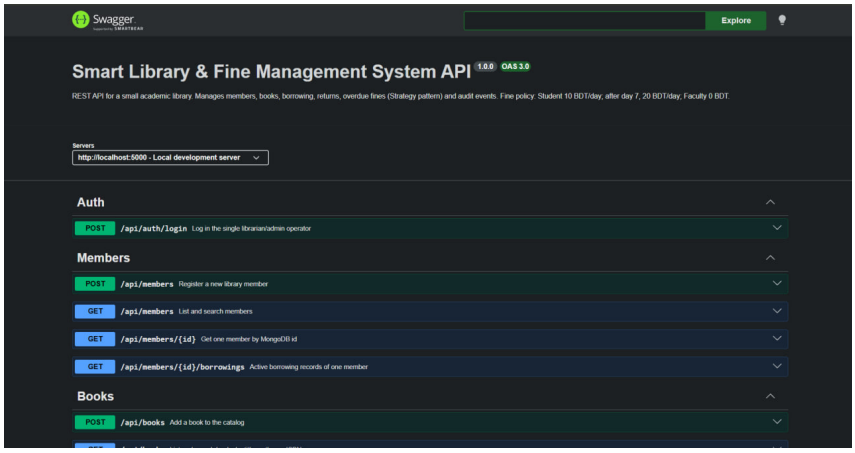


Figure 7: Interactive API Testing via Swagger UI (/api-docs)

6.2 Application User Interface Demonstration

To showcase how the documented APIs power the frontend application, the React client provides an intuitive interface across the library lifecycle:

1. **Librarian Login Gate:** Restricts system access to authenticated library personnel (Figure 8).
2. **Dashboard & Member Records:** Real-time member registration with role assignments and loan limits (Figure 9).
3. **Book Catalog:** Searchable inventory tracking total and available copies (Figure 10).
4. **Circulation & Fine Engine:** Workflow for book issues, returns, automatic overdue fine assessment, and audit logging (Figure 11).

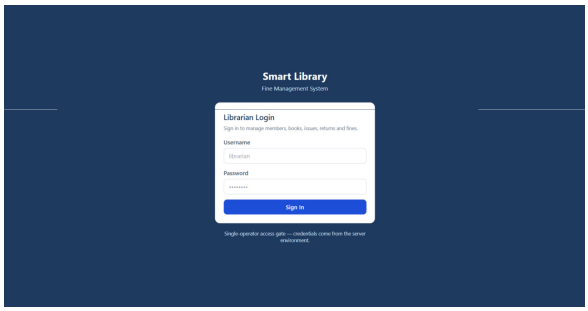


Figure 8: Librarian Login Screen

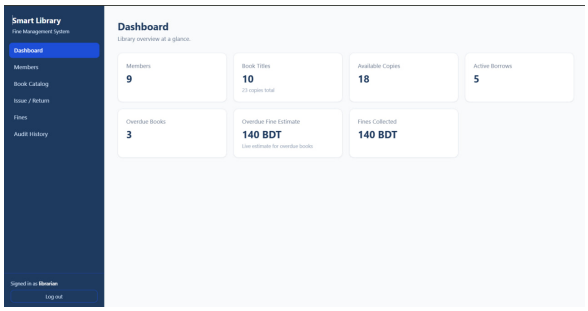


Figure 9: Dashboard and Member Management

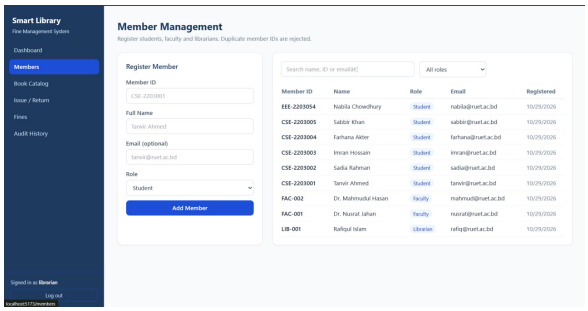


Figure 10: Book Catalog and Inventory

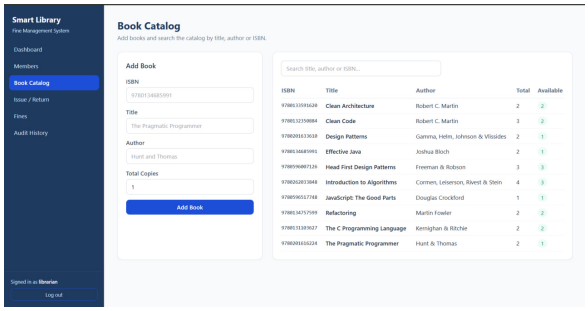


Figure 11: Circulation, Fines, and Audit Log

7 System Limitations and AI Disclosure

7.1 System Limitations

1. **Single-Operator Scope:** The current implementation focuses on internal campus library operations with a single-librarian role rather than complex multi-branch role-based access control (RBAC).

2. **Payment Gateway:** Overdue fines are mathematically calculated and tracked in the database, but external online payment gateway integration (e.g., bKash, SSLCommerz) was kept out of scope.

3. **Email Notifications:** Overdue warnings and audit logs are displayed within the web dashboard rather than through external SMS or SMTP mail servers.

7.2 AI Usage Disclosure

In compliance with the course academic integrity policy, AI tools (such as ChatGPT and GitHub Copilot) were used strictly as programming assistants for:

- Syntax guidance and debugging JavaScript asynchronous errors.
- Assisting with repetitive Jest unit test boilerplate.
- Polishing UI styling using Tailwind CSS utility classes.

All architectural design patterns, business logic implementations, and test cases were independently authored, verified through personal Git commit history, and the author is fully prepared to explain and defend every part of the codebase during the viva examination.